

TP7 : Symfony / API

Objectif

Jusqu'ici, vous avez développé une application dite "monolithique", c'est-à-dire constituée d'une seule application qui contient à la fois la partie front et la partie back. Même si cette approche a ses avantages, elle peut poser plusieurs limites, notamment en termes de scalabilité, de flexibilité et de maintenance.

En séparant le front du back, on obtient une architecture plus modulaire, facilitant les mises à jour et les évolutions indépendantes des deux parties. Cela permet aussi d'utiliser des technologies différentes pour chaque partie et de rendre le backend accessible à plusieurs types de clients (une application web et une application mobile par exemple).

Dans ce genre de configuration où le front et le back sont indépendants, il leur sera nécessaire de communiquer pour échanger des données, et cela se fera au travers d'une API.

Parenthèse sur les microservices

A partir d'un certain niveau de complexité, il peut être intéressant de moduler encore plus son application, en adoptant une architecture à base de "microservices", qui est utilisée par bons nombres d'applications aujourd'hui. Vous pouvez voir un microservice comme un composant autonome qui va être "spécialisé" dans un domaine en particulier de votre application. Par exemple, Netflix pourrait par exemple avoir un microservice qui se charge de gérer la recherche des films / séries, un autre pour gérer le lecteur vidéo, un autre pour gérer l'interface que les utilisateurs utilisent pour naviguer dans l'application, un autre pour gérer la partie "Abonnement" de chaque utilisateur, etc. ([En réalité, c'est bien plus compliqué que ça](#)).

Cette architecture offre beaucoup d'avantages. Elle facilite le développement, la maintenance et le déploiement de chaque microservice, avec généralement une équipe dédiée pour chacun d'entre eux. Néanmoins, elle est aussi beaucoup plus compliquée à maintenir !

Quand on développe une nouvelle application, on commence généralement avec une architecture monolithique, puis on envisage ensuite quelque chose de plus modulaire dans un second temps si cela devient nécessaire.

Mais revenons-en au TP. Savez-vous comment ces services vont communiquer entre eux ? Eh bien à l'aide d'APIs ! Elles occupent une place très importante dans le développement web aujourd'hui, et il est essentiel d'apprendre et les utiliser, mais aussi à les concevoir.

Définition

Une API (*Application Programming Interface*) est un ensemble de règles et de mécanismes permettant à différents logiciels de communiquer entre eux. Elle définit les méthodes et formats de données que les applications peuvent utiliser pour échanger des informations de manière

standardisée. Les API sont omniprésentes dans le développement logiciel, facilitant l'intégration entre divers systèmes et services.

Sans vous en rendre compte, vous avez en réalité déjà utilisé des API ! En effet, en JavaScript, le navigateur met à disposition [de nombreuses API Web](#), comme l'API DOM, qui vous donne accès à un ensemble d'objets de méthodes vous facilitant la modification de la structure de votre page web et du style CSS directement en JavaScript.

En fait, l'API est un concept assez général qui intervient dès lors que 2 services interagissent ensemble. Dans le cas de services web, il existe un standard, nommé REST, qui est couramment utilisé pour concevoir une API. Voici ses fondamentaux :

- Les ressources (ici, les legos) sont identifiées par des URL spécifiques, comme <https://api.example.com/product/1>, qu'on appelle des "endpoints".
- Les ressources sont manipulées à l'aide de méthodes HTTP standards (GET pour récupérer une ressource, POST pour en créer une, PUT pour la modifier, DELETE pour la supprimer).
- Les échanges de données se font via un format structuré, le plus souvent JSON, qui facilite l'interopérabilité entre différentes technologies.

Nous allons voir comment mettre en place une API au sein de notre Lego Store afin d'exposer nos legos à d'autres services web (par exemple une application mobile).

Exercice 1 - Get legos

Afin de mettre à disposition les legos à d'autres services, si on se base sur les fondamentaux d'une API REST définis plus haut, il va falloir créer un endpoint qui permettra de (1) récupérer les legos (2) dans un format structuré (3) à l'aide d'une méthode HTTP.

Commençons par le endpoint, c'est-à-dire le point d'entrée qui permettra d'exposer nos legos aux services qui souhaiteront les récupérer.

Côté Symfony, cela se traduit simplement par la création d'une nouvelle route pour notre application. D'ailleurs, toujours histoire de bien les grouper pour s'y retrouver, créez d'abord un nouveau répertoire `src/Controller/Api/` dans lequel vous placerez un nouveau `ApiLegoController` (faites bien attention au namespace). Aussi, pour bien séparer les routes de l'application "classique" de celles de l'API, faites en sorte d'ajouter un préfixe `"/api"` pour les routes des contrôleurs situés dans le dossier `src/Controller/Api/` en remplaçant le contenu de votre fichier `config/routes.yaml` par celui-ci :

```
web:
    resource:
        path: ../src/Controller/
        namespace: App\Controller
    type: attribute

api:
    resource:
        path: ../src/Controller/Api
        namespace: App\Controller\Api
    type: attribute
    prefix: '/api' # Préfixe pour les routes API
    name_prefix: 'api_' # Préfixe pour les noms des routes API
```

Nous avons dans un premier temps renommé la première entrée “web”, pour bien la différencier de la nouvelle entrée “api”, où vous noterez que nous avons ajouté un préfixe pour les routes et pour le nom des routes.

Revenons à la définition de la route dans `ApiLegoController`. Pour être conforme à l'architecture REST, une route doit respecter un format et une méthode HTTP spécifiques en fonction de ce qu'on souhaite faire avec la ressource. Les voici :

- GET `/resources` : Récupération de plusieurs ressources.
- GET `/resources/{id}` : Récupération d'une ressource.
- POST `/resources` : Création d'une nouvelle ressource.
- PUT `/resources/{id}` : Remplacement d'une ressource dans sa globalité.
- PATCH `/resources/{id}` : Remplacement de seulement quelques propriétés d'une ressource.
- DELETE `/resources/{id}` : Suppression d'une ressource.

Notez que le nom de la ressource est au pluriel, et que `{id}` doit être remplacé par n'importe quel identifiant permettant d'identifier une ressource de manière unique (généralement l'identifiant en base de données, mais cela peut aussi être le pseudo pour un utilisateur, ce qui donnerait `/users/john-doe`). À partir de ces informations, faites en sorte de définir la route pour renvoyer plusieurs legos, en précisant bien que seule la méthode GET est acceptée pour l'appeler ([jetez un oeil à la documentation](#)).

Pour ce qui est de la méthode du contrôleur qui sera appelée, il n'existe pas un format universel pour la nommer, contrairement aux routes. On essaye malgré tout de faire en sorte de respecter le même nommage au sein d'une même application. Je vous propose celui-ci :

- `all()` : Récupération de plusieurs ressources.
- `get()` : Récupération d'une ressource.
- `create()` : Création d'une nouvelle ressource.
- `update()` : Remplacement d'une ressource dans sa globalité.
- `patch()` : Remplacement de seulement quelques propriétés d'une ressource.
- `delete()` : Suppression d'une ressource.

Pour le nommage des routes, c'est la même chose : pas de convention, mais essayez de garder quelque chose de cohérent dans votre application, par exemple `{resources}.{methode}`.

Pour la route de récupération de suppression d'un lego, cela donnerait donc quelque chose comme ça :

```
#[Route('/legos/{id}', name: 'legos.delete', methods: ['DELETE'])]  
public function delete(): Response  
{  
    // ...  
}
```

Enfin, pour ce qui est du contenu de cette méthode, il faudra dans un premier temps récupérer tous les legos à renvoyer, comme c'est le cas dans le `LegoController` (comme nous ne sommes pas authentifiés, prenez bien soin de ne pas renvoyer les legos de la catégorie “premium”).

Ensuite, si vous vous rappelez bien, une méthode du contrôleur doit **toujours** retourner une réponse, sauf que cette fois, nous ne voulons pas retourner du HTML par l'intermédiaire d'un template Twig, mais la liste des legos au format JSON, qui devrait ressembler à ceci :

```
{
  "legos": [
    {
      "id": 10252,
      "name": "La coccinelle Volkswagen",
      ...
    },
    {
      "id": 10262,
      "name": "James Bond Aston Martin DB5",
      ...
    },
    ...
  ]
}
```

En vous aidant de [la documentation sur les contrôleurs](#), cherchez une manière de renvoyer quelque chose de similaire.

Tout vous paraît OK ? Bien, alors comment allons nous tester cette première route ? Pour l'instant, étant donné que c'est une simple route GET, utilisez simplement le navigateur.

Vous venez peut-être d'apprendre que votre navigateur fait en réalité des requêtes HTTP GET aux serveurs des sites auxquels vous accédez pour récupérer le code HTML / CSS / JS qui sera utilisé pour afficher les pages web.

Si vous n'avez pas d'erreur, vous constaterez qu'il y a néanmoins un problème, puisque le contenu de chaque objet est vide. En effet, par défaut, Symfony va utiliser la fonction `json_encode()` de PHP pour convertir le tableau d'objets Lego en JSON. Cependant, si cette fonction est capable de convertir des tableaux en JSON, ce n'est pas le cas des objets, qui peuvent être bien plus complexes à "sérialiser", c'est-à-dire à convertir dans une chaîne de caractères (puisque JSON, c'est avant tout une string).

De manière générale, il est possible de choisir comment une classe doit être sérialisée en implémentant la méthode magique `__serialize()` dans la classe Lego, qui retournera un tableau qui pourra ensuite être converti facilement en JSON. Cependant, dans Symfony, il existe un bundle qui offre tout un tas d'outils pour faciliter la sérialisation, notamment celle des entités.

[Suivez les instructions pour l'installer](#) et faites en sorte de l'utiliser pour renvoyer vos legos. Modifiez aussi le fichier `config/packages/framework.yaml` comme ceci afin de gérer la conversion des propriétés en camelCase / snake_case :

```
framework:
    # ...

    serializer:
```

```
name_converter: 'serializer.name_converter.camel_case_to_snake_case'
```

```
# ...
```

Vous remarquerez que vous avez maintenant une exception, comme quoi une “circular reference” a été détectée. Qu’est-ce que cela veut dire ? Eh bien quand Symfony va essayer de sérialiser vos legos, il va prendre chacune des propriétés de la classe Lego pour les ajouter au JSON final. Quand il va arriver au niveau de la collection, qui est une autre entité, à savoir LegoCollection, il va aussi essayer de la sérialiser. Sauf que celle-ci contient une propriété \$legos, qui est une collection d’objets de la classe Lego, où la collection devra à nouveau être sérialisée, etc.

En réalité, Symfony vient de nous éviter une boucle infinie ! Comment peut-on faire pour éviter ça ? Eh bien le bundle offre tout un tas d’options pour configurer la sérialisation, l’une d’entre elles est la possibilité de préciser des “groupes” de propriétés qui seront sérialisées. Essayez de créer un groupe “default” pour renvoyer toutes les propriétés de la classe Lego (même la collection), mais pas la propriété \$legos dans LegoCollection.

Vous devriez maintenant avoir une première route fonctionnelle pour récupérer les legos, ainsi que seulement les propriétés nécessaires pour chaque collection.

Exercice 2 - Pagination

Pour le moment, votre endpoint renvoie tous les legos présents en base. Cependant, si elle contenait l’ensemble des legos qui existent, vous vous doutez bien qu’il serait impensable de les récupérer et de les renvoyer en une seule fois, sinon je ne donne pas cher à la mémoire de votre serveur de production... La solution est alors de mettre en place ce que l’on appelle une pagination, qui va permettre de récupérer les legos par “paquet”.

*De manière générale, quand un endpoint renvoie un nombre indéfini de ressources, il faut **toujours** mettre en place une pagination. Gardez bien ça en tête quand vous travaillerez sur d’autres APIs (au hasard, celle de votre SAÉ).*

Il existe 2 systèmes de pagination principaux :

- La pagination par page, où le client qui utilise l’API va devoir envoyer un paramètre “page” qui permettra de définir les ressources à renvoyer.
- La pagination par curseur, qui est plus performante que celle par page sur un très grand nombre de ressources, mais plus complexe à mettre en place.

Par question de simplicité, nous allons nous pencher sur la pagination par page pour récupérer les legos.

[Si vous jetez un coup d’oeil à la documentation de Doctrine](#), vous constaterez qu’il existe une classe Paginator qui facilite la mise en place de ce type de pagination, notamment dans le cas où la requête contient des JOIN pour récupérer les ressources ainsi que certaines de leurs relations. Nous ne l’utiliserons pas ici afin que vous puissiez la mettre en place au maximum par vous-même.

Mais avant de foncer tête baissée dans le code, savez-vous déjà comment faire pour récupérer un “paquet” de legos ? Vous pourriez simplement tous les récupérer en PHP, puis utiliser une fonction comme `array_slice()` pour n’en garder qu’un certain nombre. Le problème, c’est que votre serveur

va devoir stocker temporairement dans sa mémoire vive un tableau contenant l'ensemble des legos, ce qui le fera très certainement planter avec plusieurs dizaines de millions d'entrées... Non, ce que vous voulez faire, c'est demander à votre base de vous renvoyer seulement un certain nombre de legos ! Autrement dit, de **limiter** le nombre de résultat d'un SELECT. Commencez par chercher une manière de faire ça avec MySQL, et testez que cela fonctionne bien dans phpMyAdmin.

Ensuite, il vous faudra une manière de sélectionner un paquet en particulier. Si l'utilisateur souhaite voir la page n°2, il vous faudra récupérer les legos situés après la page n°1. Faites une nouvelle recherche afin de trouver une seconde clause vous permettant de "passer" des éléments avec MySQL (autrement dit, les récupérer à partir d'un certain **offset**). Combinez-là maintenant avec la première clause permettant de sélectionner un nombre de legos définis à récupérer, et voyez comment vous pouvez récupérer les legos par "paquets".

Maintenant que vous savez comment procéder avec votre base de données, il est temps de "traduire" ça en PHP. Commencez par ouvrir votre LegoRepository, et ajoutez une méthode *findByLimitOffset()* qui, comme pour *findByCollection()*, fera bon usage du [Query Builder](#) afin de récupérer un "paquet" de legos. Cette méthode prendra en paramètre le nombre de legos qui seront retournés, ainsi que le nombre de legos à "passer".

Modifiez à présent votre ApiLegoController afin d'utiliser cette nouvelle méthode pour récupérer les legos, en essayant de jouer avec les paramètres pour vérifier que tout fonctionne bien.

Bon, c'est déjà une première étape, mais pour l'instant, il n'y a toujours pas de pagination, puisque le "client" qui utilisera votre API pourra seulement récupérer le premier paquet de legos, mais pas les suivants ! C'est maintenant que la "page" de la pagination par page entre en jeu. L'idée est que votre endpoint accepte un paramètre permettant de choisir la paquet à récupérer, ainsi qu'un second qui permettra de choisir la taille de chaque paquet.

J'imagine que vous connaissez par cœur les 2 méthodes pour transmettre des paramètres avec une requête HTTP. Mais bon, juste au cas où, voici un petit rappel :

- La première méthode consiste à inclure les paramètres directement dans l'URL de la requête. Par exemple dans la requête <https://api.google.com/search?input=hello%20world&sort=date>, il y a un paramètre "input" égal à "hello%20world" (qui est encodé, %20 correspond à un espace), et un autre paramètre "sort" égal à "date". On utilise généralement ces paramètres pour les requêtes GET, où ils permettent de donner des précisions sur ce que l'on souhaite récupérer. En PHP, ils sont accessibles grâce à la variable `$_GET`.
- La seconde méthode consiste à passer les paramètres dans le corps de la requête (le "body"). On les utilise quand on veut transmettre des données au serveur, par exemple lors de la soumission d'un formulaire. Cette méthode permet aussi de transmettre des paramètres avec des types plus complexes, comme des fichiers. On y accède avec la variable `$_POST` en PHP.

Dans votre cas de récupération de legos, quelle méthode vous paraît être la plus appropriée ?

Quand vous avez votre réponse, utilisez la documentation de Symfony afin de trouver la meilleure manière de récupérer les 2 paramètres dont vous avez besoin pour votre endpoint. Vous les nommerez respectivement "page" et "per_page", comme c'est le cas couramment pour ce genre de pagination.

Pourquoi ne pas utiliser `$_GET` ou `$_POST` pour récupérer les paramètres comme en PHP natif ? Eh bien car Symfony intègre un système de validation assez poussé permettant de vérifier que ce que le “client” correspond à ce qui est attendu par le serveur. Nous verrons cela au prochain chapitre dédié à la création d’un lego. Vous pouvez tout de même noter que Symfony effectue une première étape de validation en échappant automatiquement tous les paramètres reçus afin d’éviter, par exemple, [les injections SQL](#).

Maintenant que vous avez récupéré vos 2 paramètres, il va être nécessaire de calculer l’offset correspondant pour le passer à votre méthode `findByLimitOffset()`. Par exemple, si le nombre de legos par paquet (la valeur du paramètre `per_page`) est de 50, alors voici l’offset qui devra être calculé en fonction de la page :

- page 1 = offset 0
- page 2 = offset 50
- page 3 = offset 100
- page 4 = offset 150
- ...

Quand votre offset est calculé, passez-le en paramètre de `findByLimitOffset()` avec la valeur de `per_page`, puis testez dans votre navigateur que tout fonctionne bien.

Tout vous paraît OK ? Bravo !

Vous avez pu voir dans les 2 premiers exercices comment créer un premier type de endpoint permettant de récupérer une liste de ressources, ainsi que les bonnes pratiques pour la mise en place d’une pagination. Sachez que nous aurions pu aller encore plus loin, en ajoutant par exemple des informations à la réponse afin que l’utilisateur puisse connaître le nombre de pages maximum qu’il est possible de parcourir, mais je pense qu’il est plus intéressant de passer à la mise en place d’un nouveau type de endpoint, celui permettant de créer une ressource.

Exercice 3 - Create Lego

Intéressons-nous maintenant à la création de lego. Il est en effet courant dans une application (web ou non) de proposer des formulaires permettant à ses utilisateurs de créer certaines ressources.

Pour notre Lego Store par exemple, on pourrait imaginer une fonctionnalité permettant de soumettre une proposition de nouveau lego, qui pourrait ensuite être ajoutée au store de manière “officielle” si jamais celle-ci plaît.

Soumission de formulaire et récupération des données en PHP... C’est quelque chose que vous avez déjà fait non ? La nouveauté dans cet exercice sera, premièrement, de faire ça avec Symfony, et deuxièmement, de voir les bonnes pratiques pour faire cela en mode API.

Commençons par la définition du endpoint. Si vous reprenez les standards REST définis un peu plus haut, votre route devra accepter uniquement les requêtes de type POST, et respecter le format `/ressources` (au pluriel).

Toujours dans votre `ApiLegoController`, définissez cette route ainsi que la méthode du contrôleur associée **en prenant bien soin de respecter le nommage qui a aussi été défini plus haut**.

Pour ce qui est du contenu de cette méthode, vous allez dans un premier temps avoir besoin de récupérer les données contenues dans la requête. Pour cela, il est possible d'injecter l'objet *Request* de Symfony, qui offre tout un tas de méthodes pour récupérer des informations sur la requête, dont les paramètres ([plus d'infos sur la documentation](#)). Qui dit paramètres, dit obligatoirement validation. C'est une règle élémentaire dans le web : "Never trust user input", ne faites jamais confiance aux données envoyées par l'utilisateur !

Même s'il est souvent intéressant de faire une validation côté front pour que la navigation soit plus fluide pour l'utilisateur, elle ne doit jamais remplacer une validation côté back ! En effet, un utilisateur mal intentionné peut très bien faire des requêtes directement à votre serveur sans passer par votre formulaire.

À vous de jouer. Faites en sorte de récupérer toutes les informations du nouveau lego dans votre contrôleur, puis de valider qu'elles sont toutes différentes de null. Si ce n'est pas le cas, vous pouvez "throw" une exception avec un message approprié. Je vous conseille d'utiliser *HttpException*, qui est inclus avec Symfony, et qui sera automatiquement convertie en une réponse HTTP appropriée :

```
throw new HttpException(422, 'The name is required.');
```

N'acceptez pas de paramètre "id", il sera automatiquement généré par MySQL lorsque vous insérerez votre lego.

Pour ce qui est des 2 images, le processus est un peu plus complexe. L'objectif va être, comme pour les legos existants, de récupérer les fichiers transmis avec la requête, de les déplacer avec les autres images de legos (donc dans /public/images/) et de seulement stocker le nom des fichiers en base de données.

Commencez par récupérer les images dans 2 variables séparées. Vous constaterez en lisant la documentation que Symfony vous fournit des instances d'une classe *UploadedFile*, qui contient plusieurs méthodes permettant de les manipuler. Ensuite, [en vous aidant de cet exemple de la documentation](#), créez vous-aussi un service *FileUploader* que vous allez pouvoir injecter dans votre contrôleur et utiliser afin de générer un nom de fichier approprié et déplacer les images dans le bon dossier. D'ailleurs, concernant le dossier, vous constaterez qu'il a lui aussi été injecté dans le constructeur du *FileUploader*. Comment est-ce possible ? Eh bien en définissant manuellement le service et le paramètre dans le fichier *config/services.yaml*. De cette manière, vous avez la possibilité de récupérer des variables fournies par Symfony, comme le répertoire dans lequel se situe l'application. Pour cela, vous pouvez faire quelque chose comme ça :

```
# config/services.yaml
services:
    # ...

    App\Service\FileUploader:
        arguments:
            $targetDirectory: '%kernel.project_dir%/public/images'
```


Maintenant que vous avez récupéré et “validé” les paramètres de votre requête, il est temps de passer à la création du Lego. Pour cela, vous pouvez vous inspirer de [cet exemple de la documentation](#), qui vous montre comment créer une entité de manière générale.

Il vous reste une dernière étape : retourner une réponse adaptée pour informer le client que tout s’est bien passé. Lors de la création d’une ressource, il y a 2 écoles : soit vous retournez l’entité qui vient tout juste d’être créée, soit vous retournez simplement une réponse “vide”. Personnellement, j’ai tendance à préférer la deuxième option, votre route a déjà assez de choses à gérer avec la création. De plus, avec un client utilisant une bibliothèque qui gère un “state” (comme React), il est généralement courant de refaire un appel au endpoint de récupération des ressources plutôt que de gérer le retour de la ressource après sa création.

Dans tous les cas, que l’on choisisse de retourner la ressource ou non, le code HTTP associé à la création d’une ressource avec succès est 201. Voilà une manière de renvoyer une réponse vide avec ce code :

```
return new Response('', Response::HTTP_CREATED);
```

Essayez maintenant de tester votre endpoint. Cette fois-ci, vous n’allez pas pouvoir utiliser votre navigateur, qui envoie simplement des requêtes GET. Je ne vais pas non plus vous demander de créer un formulaire sur votre Lego Store, c’est quelque chose que vous savez déjà faire. Non, à la place, vous allez utiliser un outil qui est dédié au testing d’API. Il en existe plusieurs, l’un des plus connus et que j’utilise personnellement est Postman, que je vous propose d’en faire de même.

[Après l’avoir installé](#) (vous n’avez pas besoin de créer un compte), commencez par créer une première requête pour faire appel à l’endpoint GET de récupération de legos. Pour l’envoyer, cliquez sur le bouton à droite, le résultat devrait s’afficher en bas. Pour ajouter les paramètres *page* et *per_page*, vous pouvez utiliser l’onglet dédié en dessous de votre requête. Postman devrait les formater automatiquement en ajoutant *?param=valeur* à votre URL.

Si tout est fonctionnel, créez une nouvelle requête, cette fois-ci de type POST, afin de tester la création de lego. Essayez simplement de l’envoyer sans aucun paramètre afin de voir si le serveur vous renvoie une réponse 422 avec le message que vous avez choisi.

Il y a un problème, vous ne trouvez pas ? Par défaut, Symfony renvoie réponse sous forme de HTML (vous pouvez la visualiser à l’aide de l’onglet “Preview” en bas). Étant donné que cet endpoint sera appelé par une requête AJAX qui s’attend à recevoir du JSON, ce n’est pas l’idéal. Heureusement, Symfony permet de changer ça très facilement. Dans votre *ApiLegoController*, ajoutez le paramètre “format” à la définition de votre route comme ceci :

```
#[Route('/legos', name: 'legos.create', methods: ['POST'], format: 'json')]
```

Réessayez de soumettre votre requête et constatez que Symfony vous renvoie désormais un JSON avec des informations sur l’erreur. Profitez-en pour ajouter le paramètre “format” à votre route de récupération de legos, et **pensez à bien l’ajouter sur les prochaines routes de votre API qui renverront du JSON !**

Vous pouvez maintenant ajouter des paramètres au body / payload de votre requête. Pour cela, cliquez sur l’onglet “Body” sous l’URL, puis “form-data”. Pour les fichiers, vous pouvez faire un hover sur le nom des paramètres et sélectionner “File”.

The screenshot shows a web client interface for a POST request to `http://localhost:8787/api/legos`. The request body is configured as `form-data` with the following fields:

Key	Value
☒ pieces	1000
☒ boximage	cover.jpg
☒ legoimage	cover.jpg
☐ collection_id	2

A dropdown menu is open over the `boximage` and `legoimage` fields, showing options: `File` (selected), `Text`, and `File`.

À vous de jouer pour faire fonctionner votre route de création de lego ! Si vous avez des erreurs, lisez bien les messages de Symfony, et n'hésitez pas à utiliser les fonctions `dump()` et `dd()` afin de déboguer vos variables.

Exercice 4 : Create Lego, mais plus propre

Prenez le temps d'analyser un peu tout ce qui est fait dans la méthode `create()` de votre `LegoController`. Entre la définition de la route, la récupération et la validation des paramètres, la création du lego et enfin la réponse, vous ne trouvez pas que cela fait beaucoup ?

Une bonne pratique dans le développement informatique est d'avoir des classes et des méthodes ayant une seule responsabilité, ceci dans le but d'améliorer la lisibilité, la compréhension et la flexibilité du code (c'est en réalité le S des principes SOLID, [lisez cet article si ça vous intéresse](#)).

Dans le cas des méthodes d'un contrôleur, vous pouvez considérer que leur responsabilité est, grand maximum, de récupérer des paramètres, de faire appel à un ou plusieurs services, et de renvoyer une réponse. Leur objectif n'est pas d'effectuer un quelconque traitement "métier", mais de faire appel à d'autres classes et méthodes qui s'en chargeront.

Commençons par la création du lego. Pensez-vous vraiment que le rôle d'un contrôleur est de créer des entités ? Bien évidemment, vous vous dites que non, et vous avez parfaitement raison ! Si vous reprenez ce qui vous a été dit plus haut, un contrôleur fait simplement office de relais entre plusieurs "services", et ne comprend pas, ou peu, de réels traitements (le code "métier").

Si vous vous souvenez bien, au TP3, vous avez créé un lego directement dans votre contrôleur, avant de déplacer ce code vers un `LegoService`, ce qui rendait votre contrôleur plus léger et plus clair, tout en permettant de facilement réutiliser le code de récupération d'un lego ailleurs dans votre application. Finalement, vous avez installé Doctrine et fait usage du `LegoRepository` à la place, ce qui a rendu le `LegoService` obsolète.

Pour la création de lego, c'est un peu la même chose. ça serait quand plus propre de laisser un service se charger de la création de l'entité en tant que telle !

Si vous aviez supprimé le *LegoService*, recréez-le et placez-le dans le répertoire *src/Service/*, puis faites en sorte de l'utiliser dans votre *LegoController* et votre *ApiLegoController* pour récupérer les legos.

Ajoutez une méthode publique *create()* dans le *LegoService* qui prendra en paramètres les différentes informations du lego, puis placez-y le code que vous aviez dans le *ApiLegoController* qui vous permettait de le créer, ainsi que l'upload des 2 images. Concernant les services *EntityManagerInterface* et *FileUploader*, vous n'en avez maintenant plus l'utilité dans votre contrôleur, et vous pouvez déplacer leurs injections dans le constructeur de votre *LegoService*. Vous devrez bien évidemment créer deux nouvelles propriétés (privées) afin d'y avoir accès au sein de la méthode *create()*. Enfin, mettez à jour votre contrôleur afin d'injecter et d'utiliser le *LegoService*. C'est quand même plus propre vous ne trouvez pas ?

À présent, concentrons-nous sur le reste du code de la méthode *create()* du *ApiLegoController*. Dans Symfony, il est facilement possible de déléguer la récupération des paramètres de la requête et leur validation, afin que le contrôleur ait seulement besoin de les rediriger vers un service qui fera le traitement nécessaire (en l'occurrence, le *LegoService*). En vous aidant de la [documentation](#), faites en sorte de mapper les paramètres reçus contenant les informations du lego dans une classe *CreateLegoPayload*, que vous pouvez placer dans un répertoire *src/Dto/Payload/*.

Pourquoi dans ce dossier précisément ? DTO veut dire "Data Transfer Object". Ce sont des objets qui ont pour but de contenir et transmettre des données au sein d'une application. Utilisés à bon escient, ils sont plutôt pratiques, car ils vous permettent de ne passer qu'un seul paramètre à vos méthodes au lieu d'en avoir un pour chaque propriété de lego. Pour ce qui est du dossier Payload, c'est simplement car vous pourriez potentiellement avoir d'autres DTO dans votre application qui contiendrait autres choses que des payloads de requête HTTP.

De retour dans votre contrôleur, vous avez à présent besoin d'injecter un objet *\$payload* de type *CreateLegoPayload* dans votre contrôleur en utilisant le bon attribut afin que Symfony fasse le mapping des paramètres reçus automatiquement.

Malheureusement, Symfony ne supporte pas la possibilité de mapper des fichiers dans des DTO de cette manière. Vous devrez donc récupérer les 2 images de la même manière que précédemment dans votre contrôleur. Cependant, pour que ça reste propre, vous pouvez très bien les ajouter à votre *CreateLegoPayload* à l'aide de 2 setters !

Une fois que c'est fait, il ne vous reste plus qu'à passer ce *\$payload* à *LegoService::create()* et à l'utiliser à bon escient afin que la création de lego reste fonctionnelle.

Si tout vous paraît bon, vous pouvez revenir sur Postman et vérifier que tout fonctionne bien. Pensez aussi à tester la création d'un lego en omettant un des paramètres et vérifiez que Symfony vous renvoie une 422 avec un message approprié, comme c'était le cas précédemment.

Bon, votre création de lego fonctionne toujours, mais il ne manquerait pas quelque chose ? Qu'est-ce qu'il se passe si vous essayez de passer un nombre entier pour votre description de lego ? ou bien une collection qui n'existe pas ? Peut-être que votre serveur vous renverra des erreurs 500, ou, pire, que votre lego sera bien inséré en base avec une collection à NULL !

Ce qu'il vous manque, c'est bien évidemment la validation. Sachez que là encore, il existe un composant dédié dans Symfony, nommé "Validator", qui contient tout un tas de règles (*constraint* en anglais) facilitant grandement le processus de validation. Vous commencez à avoir l'habitude : à partir de [la documentation](#), installez ce composant et cherchez comment ajouter des "*constraints*" à chaque propriété de votre *CreateLegoPayload* qui respecte les règles suivantes :

- nom : chaîne de caractères, maximum 30 caractères.
- description : chaîne de caractères, maximum 500 caractères.
- prix : nombre flottant.
- pièces : nombre entier.
- lego image : fichier image (jpg, png uniquement)
- box image : fichier image (jpg, png uniquement)
- collection : une collection valide. (je vous conseille de créer [une contrainte personnalisée](#) qui fera appel à votre *LegoCollectionRepository* pour récupérer les collections valides).

Bon courage !

Exercice 5 : Authentification

Il nous reste un dernier point à aborder, celui de l'authentification. En effet, dans votre Lego Store, il est possible pour les utilisateurs de se connecter afin d'accéder à des legos "premium". Pour répondre à cette problématique, vous aviez mis en place une authentification par session, qui est particulièrement adaptée dans le cas d'une application web monolithique, puisque le navigateur s'occupe automatiquement de stocker et de faire passer le cookie de session à chaque requête. Cependant, dans le cas d'une API, les "clients" ne sont pas forcément des navigateurs capables de gérer des cookies. Cela peut par exemple être des applications mobiles, d'autres serveurs, ou même une console de jeu. C'est pourquoi les API proposent généralement de s'authentifier d'une autre manière : grâce à des tokens. Ces tokens sont en réalité des chaînes de caractères sécurisées qui permettent au serveur d'identifier un utilisateur lors d'une requête à l'API. Voyons comment mettre ça en pratique dans Symfony.

Si vous ouvrez [la documentation du Security Bundle](#), vous constaterez qu'il existe plusieurs méthodes d'authentification. Lors des TP, vous aviez utilisé "Form Login" qui permet à un utilisateur de s'authentifier avec un cookie de session en soumettant un formulaire directement présent dans l'application Symfony.

Dans le cas d'une API, 2 méthodes paraissent plus appropriées : JSON Login et Access Token (qui n'est bizarrement pas référencé dans la liste des différentes méthodes d'authentification, mais qui est pourtant bien présente sur la page. Vous pouvez la retrouver dans le menu à gauche de la documentation).

Si JSON Login paraît particulièrement adaptée dans votre cas, elle fournit en réalité une authentification via un cookie de session. Cependant, si vous regardez les exemples, après une authentification réussie sur un endpoint de login, un token est bien créé et retourné, qui laisse sous-entendre qu'il est possible de s'authentifier avec dans un second temps, mais ce n'est à priori pas le cas, du moins pas sans la méthode Access Token.

La meilleure solution consiste en réalité à faire un mix des deux, c'est-à-dire utiliser JSON Login uniquement sur votre route de login, où l'email et le mot de passe seront transmis par votre front afin de récupérer un token, puis Access Token pour le reste de votre application.

Pour mettre ça en pratique, le plus propre est de mettre en place deux firewalls distincts, nous allons commencer par celui qui sera actif pour la connexion. Modifiez votre firewall “main” dans `security.yaml` pour ajouter une nouvelle méthode d’authentification de type JSON Login. Créez à présent un nouveau `SecurityController` dans `src/Controller/Api/` et ajoutez une méthode `login()` qui sera appelé lors d’un appel POST à `/api/login`. N’oubliez pas de lui préciser le format “json” comme pour les autres routes, et surtout de bien renseigner la valeur de la propriété `check_path` utilisée par JSON Login dans le `security.yaml` en utilisant le nom de votre route.

Si vous vous souvenez bien, vous aviez indiqué dans votre fichier `config/routes.yaml` que toutes les routes (et leurs noms) présentes dans le dossier `src/Controller/Api` soient automatiquement précédées d’un préfixe “/api” (et “api_” pour leurs noms). Faites attention à ne pas les ajouter une seconde fois dans la définition de votre route de login dans le contrôleur.

Injectez maintenant votre utilisateur comme ceci et faites en sorte de le “dumper” pour voir si tout est OK :

```
public function login(
    #[CurrentUser()] User $user,
): Response
{
    dd($user);
}
```

À l’aide de Postman, essayez d’appeler cette nouvelle route `/api/login` en envoyant de bonnes mais aussi de mauvaises informations de connexion, afin de voir ce que Symfony répond par défaut dans ce genre de cas.

Si vous regardez la documentation de JSON Login, vous verrez que les paramètres par défaut attendus par Symfony pour se connecter sont “username” et “password”. Si vous le souhaitez, vous pouvez les changer dans votre `security.yaml` [à l’aide des propriétés `username\_path` et `password\_path`](#).

Et voilà, en modifiant simplement quelques lignes dans un fichier de configuration, Symfony vous permet d’ajouter une nouvelle méthode d’authentification à votre application. Le plus pratique, c’est que vous n’avez pas besoin de vous soucier de la manière dont votre utilisateur s’est connecté, puisque Symfony vous fournira dans tous les cas un objet `User` !

Passons à la suite. Maintenant que vous avez votre `$user`, la prochaine étape va être de générer un token et de le renvoyer au front, qui pourra l’utiliser pour se connecter aux autres endpoints nécessitant une authentification. L’idée est d’avoir maximum un token par utilisateur. Pour stocker ces tokens en base, je vous conseille d’utiliser **une entité et une table dédiée**. Vous pourrez ensuite faire une relation avec votre entité `User` afin de récupérer facilement le token associé. Réfléchissez-bien au type de relation à définir, un utilisateur pourra avoir au maximum un seul token.

Une fois que c’est fait, il vous faut maintenant créer un nouveau token (et éventuellement supprimer l’ancien) lorsqu’un utilisateur se connecte avec succès sur votre nouvelle route de login. Vous pouvez utiliser ceci pour générer sa valeur : `bin2hex(random_bytes(32))`.

Vous n’avez bien évidemment pas pensé une seule seconde à écrire tout ce code dans votre `SecurityController`, et vous vous êtes dit qu’un service serait bien plus approprié n’est-ce-pas ?

Une fois le token inséré en base, il vous suffit de renvoyer sa valeur au front, qui pourra le stocker et l'utiliser pour s'authentifier.

Si vous souhaitez ajouter plus de sécurité, voici quelques pistes :

- Comme pour les mots de passe, vous pouvez hacher vos tokens avant de les insérer en base avec la méthode `hash('sha256', $rawToken)` par exemple. De cette manière, une fois qu'un token est généré, il ne sera plus jamais possible de récupérer sa valeur. Elle sera seulement renvoyée par `/login` lorsque l'utilisateur essaiera de se connecter.
- Vous pouvez ajouter une colonne permettant de gérer une date d'expiration pour vos tokens.

Maintenant que votre route de connexion est fonctionnelle, voyons comment faire en sorte d'utiliser le token pour se connecter. À l'aide de la documentation, modifiez votre firewall "main" pour ajouter [une authentification par Access Token](#).

Vous aurez besoin d'ajouter une classe `AccessTokenHandler`, qui prendra en paramètre le token envoyé par le front et vérifiera sa validité. Si c'est le cas, il faudra ensuite retourner une instance de `UserBadge`, qui attend en paramètre **la propriété permettant d'identifier les utilisateurs qui a été configuré dans security.yaml, au niveau du provider** (l'email ou le username dans votre cas). Attention à ne pas utiliser l'id, comme pourrait laisser croire la documentation.

Vous vous demandez peut-être comme Symfony sait où récupérer le token dans votre requête ? Comme précisé dans la [documentation](#), Symfony va par défaut chercher le token dans un header avec ce format : "Authorization: Bearer \$votreToken", qui est généralement le standard pour ce genre d'authentification.

Dans Postman, vous avez un onglet "Authorization" qui vous donne la possibilité d'ajouter un "Bearer Token" à vos requêtes. Vous pouvez aussi faire en sorte que ce header soit ajouté automatiquement à toutes vos requêtes, en les groupant au sein d'une même "Collection", qui contient aussi un onglet "Authorization". Vous devrez ensuite configurer chaque requête pour utiliser le type d'authentification "Inherit auth from parent" au lieu de "Bearer Token".

Tout vous paraît OK ? Pour en être sûr, vérifiez que :

- Les legos "Premium" sont bien récupérés uniquement si vous êtes identifiés avec le token.
- La création d'un lego "Premium" fonctionne uniquement pour les utilisateurs authentifiés.

Attention, car Postman intercepte parfois les cookies qui sont renvoyés par les endpoints et les ajoute automatiquement lors de vos prochains requêtes, ce qui aura pour effet de vous authentifier automatiquement à l'aide du cookie de session (comme votre navigateur). Pour vérifier si c'est le cas, cliquez sur le bouton "Cookies" situé juste en-dessous de celui pour envoyer vos requêtes, et, s'il est présent, supprimez le domaine "localhost" et tous les cookies associés.

Pour résumer

Durant ce TP, vous avez découvert (ou redécouvert) ce qu'était une API, et comment en implémenter une au sein d'une application Symfony à l'aide des standards REST. Concrètement, cela s'est traduit par :

- La mise en place premier endpoint pour récupérer des legos au format JSON, en mettant en place une pagination.

- L'implémentation d'un second endpoint de création de lego, en ayant au préalable effectué une validation des informations reçues.
- La possibilité de se connecter et de s'authentifier à l'aide d'un token sécurisé.

Félicitations, vous avez terminé tous les TP 🎉